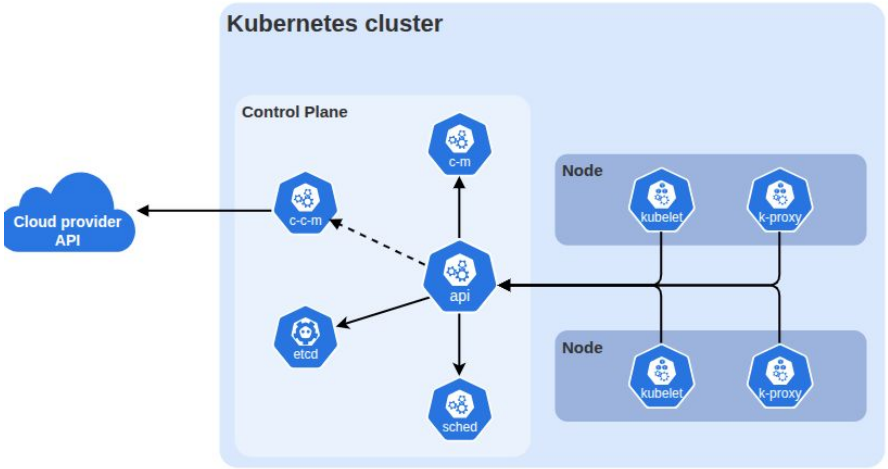
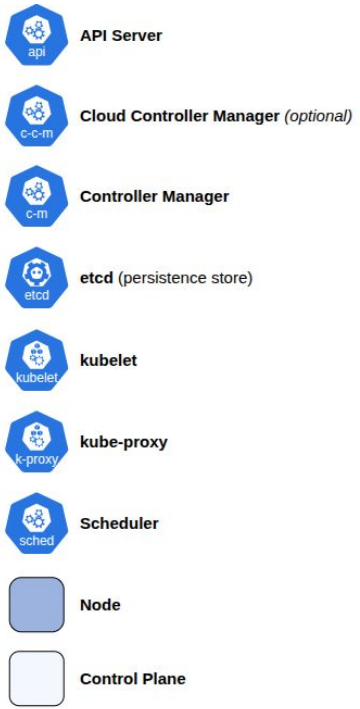
Gerência de configuração

Continuação ao Kubernetes (parte 2)

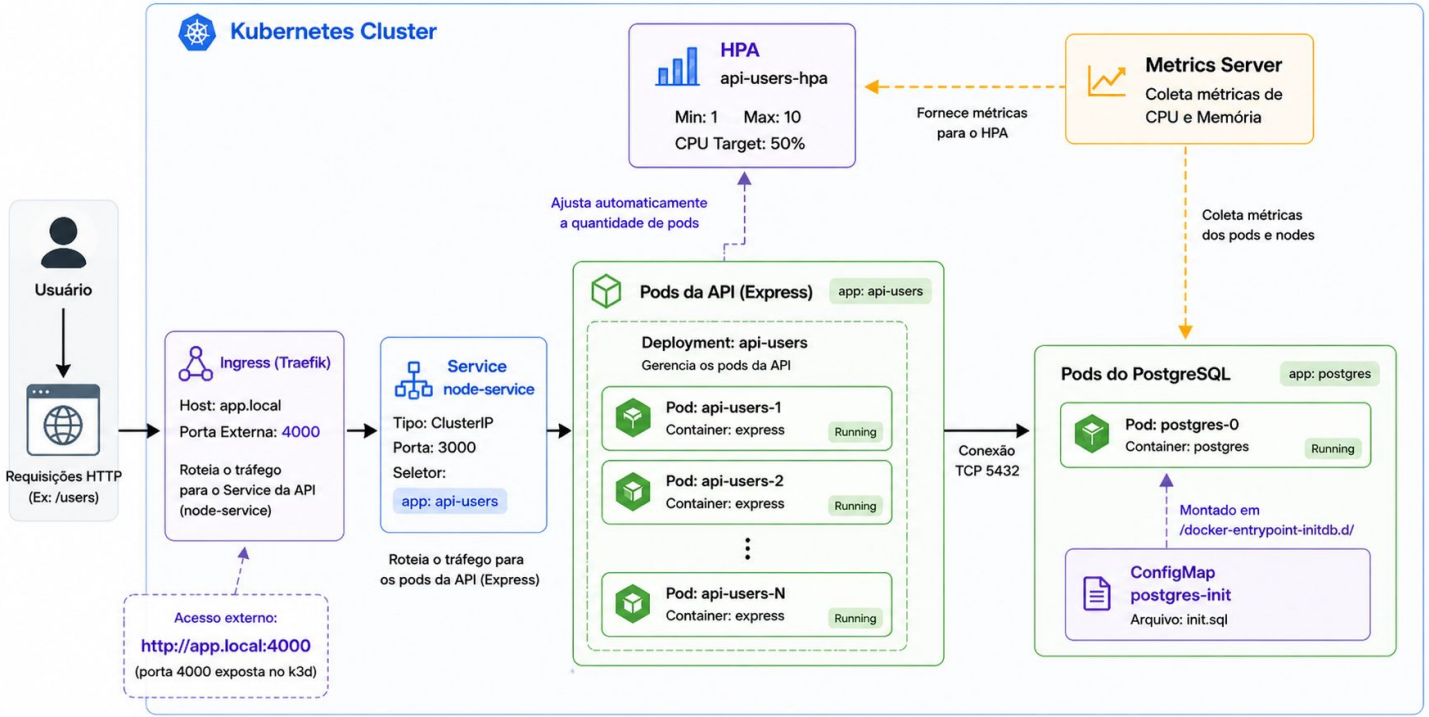
Prof. Diego Cardoso Borda Castro

Componentes para K8s



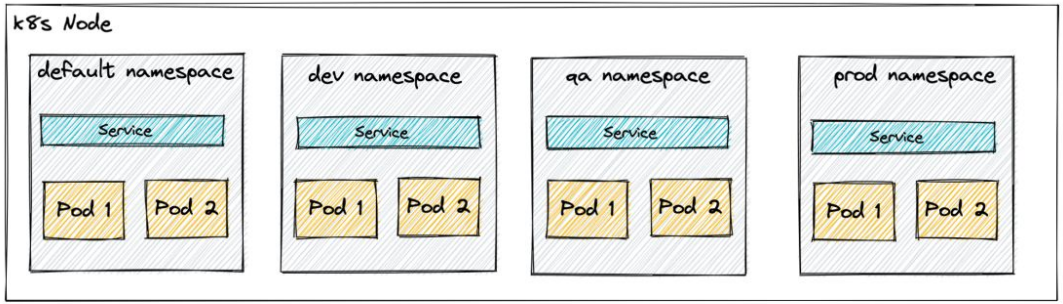


Estrutura do último projeto



Namespaces

- Separação lógica dentro do cluster
- Permite:
  - isolamento
  - organização
  - quotas
  - RBAC separado

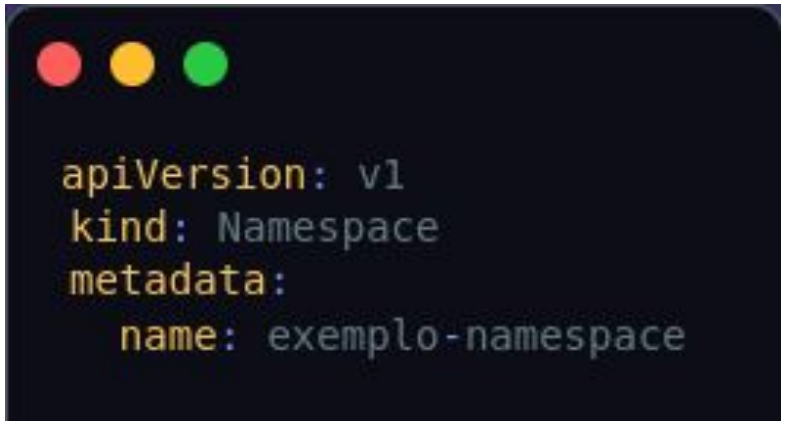


Namespaces

- Por padrão, o Kubernetes vem com alguns Namespaces
  - default: O Namespace padrão para recursos que não têm um Namespace específico
  - kube-system: Utilizado para recursos criados pelo próprio Kubernetes
  - kube-public: Um Namespace especial que é acessível publicamente
  - kube-node-lease: Usado para gerenciar o ciclo de vida dos nodes

Namespaces

```
kubectl get namespaces
kubectl config set-context --current --namespace=<novo-namespace>
kubectl create namespace exemplo-namespace-2
```



```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: exemplo-namespace
```

- A boa prática sempre manda trabalhar com namespaces para conseguir segregar cada conjunto de recursos

Quotas

- Serve para limitar o consumo de recursos dentro de um namespace
  - kubectl get resourcequota -n dev
- Quota vs deploy
  - resources no Deployment
    - Limita recursos para os pods
  - ResourceQuota
    - Limita o namespace inteiro

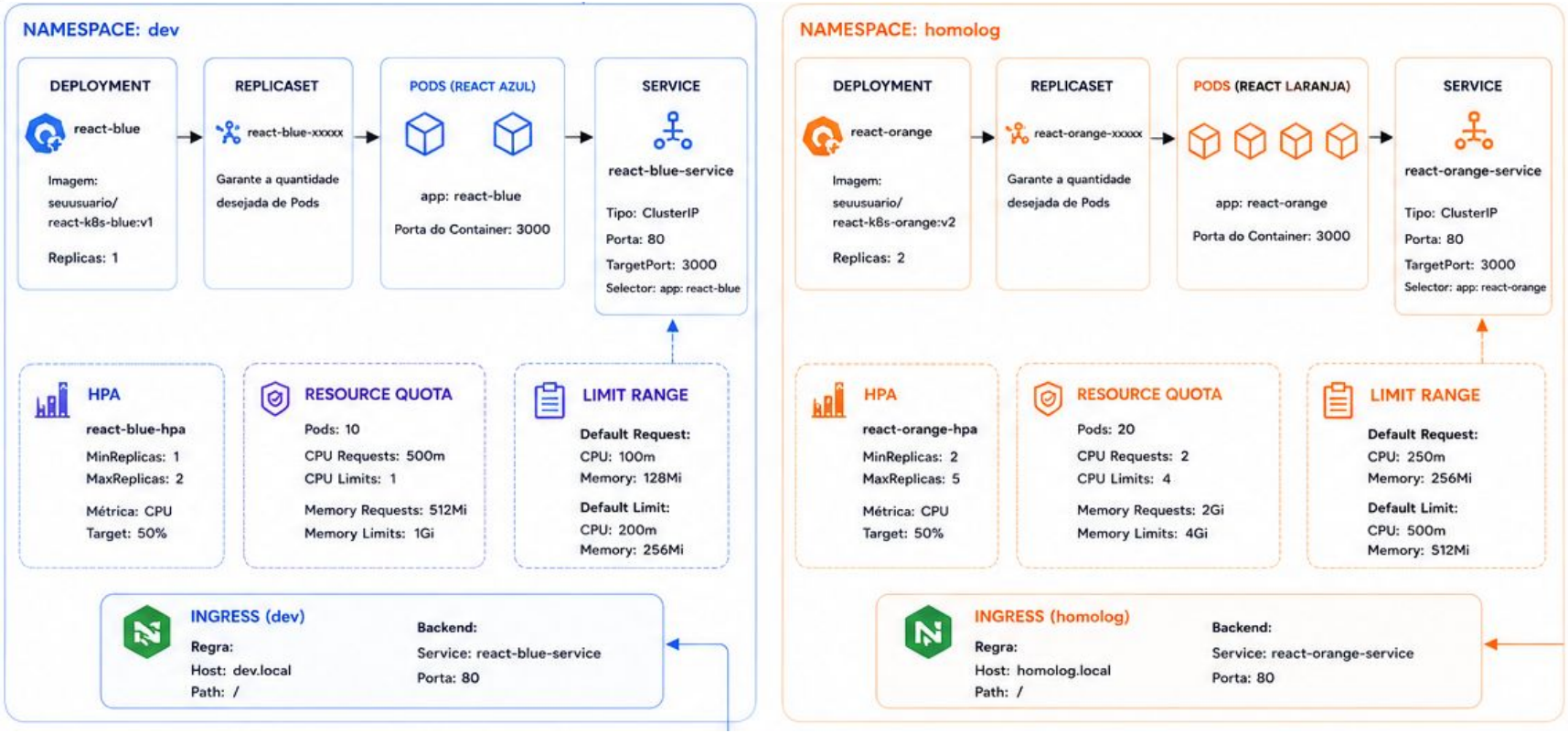
```yaml
apiVersion: v1
kind: ResourceQuota

metadata:
  name: dev-quota
  namespace: dev

spec:
  hard:
    pods: "2"

    requests.cpu: "500m"
    requests.memory: "512Mi"

    limits.cpu: "1"
    limits.memory: "1Gi"
```


NAMESPACE: dev
DEPLOYMENT
react-blue
Imagem:
seuusuario/
react-k8s-blue:v1
Replicas: 1
REPLICASET
react-blue-xxxxx
Garante a quantidade
desejada de Pods
PODS (REACT AZUL)
app: react-blue
Porta do Container: 3000
SERVICE
react-blue-service
Tipo: ClusterIP
Porta: 80
TargetPort: 3000
Selector: app: react-blue
HPA
react-blue-hpa
MinReplicas: 1
MaxReplicas: 2
Métrica: CPU
Target: 50%
RESOURCE QUOTA
Pods: 10
CPU Requests: 500m
CPU Limits: 1
Memory Requests: 512Mi
Memory Limits: 1Gi
LIMIT RANGE
Default Request:
CPU: 100m
Memory: 128Mi
Default Limit:
CPU: 200m
Memory: 256Mi
INGRESS (dev)
Regra:
Host: dev.local
Path: /
Backend:
Service: react-blue-service
Porta: 80
NAMESPACE: homolog
DEPLOYMENT
react-orange
Imagem:
seuusuario/
react-k8s-orange:v2
Replicas: 2
REPLICASET
react-orange-xxxxx
Garante a quantidade
desejada de Pods
PODS (REACT LARANJA)
app: react-orange
Porta do Container: 3000
SERVICE
react-orange-service
Tipo: ClusterIP
Porta: 80
TargetPort: 3000
Selector: app: react-orange
HPA
react-orange-hpa
MinReplicas: 2
MaxReplicas: 5
Métrica: CPU
Target: 50%
RESOURCE QUOTA
Pods: 20
CPU Requests: 2
CPU Limits: 4
Memory Requests: 2Gi
Memory Limits: 4Gi
LIMIT RANGE
Default Request:
CPU: 250m
Memory: 256Mi
Default Limit:
CPU: 500m
Memory: 512Mi
INGRESS (homolog)
Regra:
Host: homolog.local
Path: /
Backend:
Service: react-orange-service
Porta: 80

Criando nosso exemplo



```yaml
apiVersion: v1
kind: Namespace

metadata:
  name: dev

---
apiVersion: v1
kind: Namespace

metadata:
  name: homolog
```

```
k3d cluster create meucluster \
  --servers 1 \
  --agents 2 \
  -p "4000:80@loadbalancer"
```

```
kubectl cluster-info
kubectl get nodes
```

```
kubectl apply -f namespaces.yaml
```

```yaml
apiVersion: apps/v1
kind: Deployment

metadata:
  name: react-blue
  namespace: dev

spec:
  replicas: 1

  selector:
    matchLabels:
      app: react-blue

  template:
    metadata:
      labels:
        app: react-blue

    spec:
      containers:
        - name: react-blue

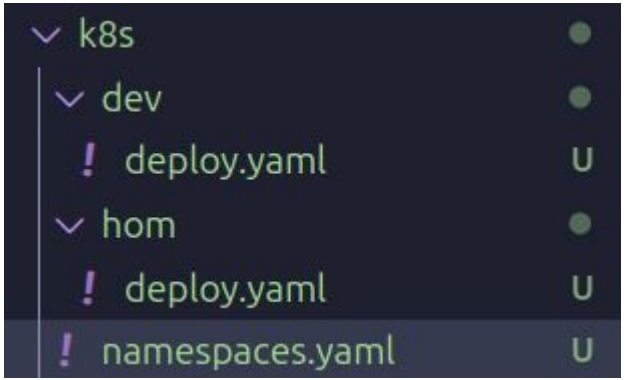
          image: diegocbcastroo/react-k8s-blue:v1

          ports:
            - containerPort: 3000

          resources:
            requests:
              cpu: "100m"
              memory: "64Mi"

            limits:
              cpu: "200m"
              memory: "128Mi"
```

Orange



- Crie a estrutura acima
  - kubectl apply -Rf ./k8s
  - Em hom
    - Troque para 2 as réplicas

```yaml
apiVersion: v1
kind: Service

metadata:
  name: react-blue-service
  namespace: dev

spec:
  selector:
    app: react-blue

  ports:
    - port: 80
      targetPort: 3000

  type: ClusterIP
```

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler

metadata:
  name: react-blue-hpa
  namespace: dev

spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: react-blue

  minReplicas: 1
  maxReplicas: 2

  metrics:
    - type: Resource

      resource:
        name: cpu

        target:
          type: Utilization
          averageUtilization: 50
```

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress

metadata:
  name: react-ingress
  namespace: dev

spec:
  ingressClassName: traefik

  rules:
    - host: dev.local

      http:
        paths:
          - path: /
            pathType: Prefix

            backend:
              service:
                name: react-blue-service

                port:
                  number: 80
```

kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

- sudo nano /etc/hosts
  - 127.0.0.1 dev.local
  - 127.0.0.1 homolog.local

- kubectl get pods -n dev
- kubectl get pods -n homolog

```yaml
apiVersion: v1
kind: ResourceQuota

metadata:
  name: dev-quota
  namespace: dev

spec:
  hard:
    pods: "10"

    requests.cpu: "500m"
    requests.memory: "512Mi"

    limits.cpu: "1"
    limits.memory: "1Gi"
```

- kubectl get resourcequota -n dev
- Até agora temos
  - 2 nodes
  - Dev
    - 2 pods (max)
  - homolog
    - 5 pods (max)

```yaml
apiVersion: v1
kind: ResourceQuota

metadata:
  name: dev-quota
  namespace: dev

spec:
  hard:
    pods: "10"

    requests.cpu: "500m"
    requests.memory: "512Mi"

    limits.cpu: "1"
    limits.memory: "1Gi"
```

- kubectl get resourcequota -n dev
- Até agora temos
  - 2 nodes
  - Dev
    - 2 pods (max)
  - homolog
    - 5 pods (max)
- Problemas
  - 2 node com vários
  - Compartilhamento entre dev e hom

while true; do curl http://dev.local:4000 > /dev/null; done

~ 83x19

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   644  100   644    0     0   160k      0 --:--:-- --:--:-- --:--:--  209k
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   644  100   644    0     0   177k      0 --:--:-- --:--:-- --:--:--  209k
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   644  100   644    0     0   118k      0 --:--:-- --:--:-- --:--:--  125k
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   644  100   644    0     0   191k      0 --:--:-- --:--:-- --:--:--  209k
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   644  100   644    0     0   192k      0 --:--:-- --:--:-- --:--:--  209k
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   644  100   644    0     0   182k      0 --:--:-- --:--:-- --:--:--  209k
```

~ 86x19

```
kubectl get hpa -n dev -w
NAME              REFERENCE                    TARGETS       MINPODS   MAXPODS   REPLICAS
AGE
react-blue-hpa    Deployment/react-blue    cpu: 0%/50%   1         2         1
16m
react-blue-hpa    Deployment/react-blue    cpu: 21%/50%   1         2         1
 17m
react-blue-hpa    Deployment/react-blue    cpu: 0%/50%    1         2         1
 17m
react-blue-hpa    Deployment/react-blue    cpu: 107%/50%   1         2         1
  17m
react-blue-hpa    Deployment/react-blue    cpu: 137%/50%   1         2         2
  18m
react-blue-hpa    Deployment/react-blue    cpu: 109%/50%   1         2         2
  18m
react-blue-hpa    Deployment/react-blue    cpu: 75%/50%    1         2         2
  18m
```

~ 85x20

```
100   644  100   644    0     0   213k      0 --:--:-- --:--:-- --:--:--  314k
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   644  100   644    0     0   208k      0 --:--:-- --:--:-- --:--:--  314k
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   644  100   644    0     0   152k      0 --:--:-- --:--:-- --:--:--  209k
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   644  100   644    0     0   110k      0 --:--:-- --:--:-- --:--:--  125k
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   644  100   644    0     0   178k      0 --:--:-- --:--:-- --:--:--  209k
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   644  100   644    0     0   160k      0 --:--:-- --:--:-- --:--:--  209k
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   644  100   644    0     0   204k      0 --:--:-- --:--:-- --:--:--  314k
```

~ 85x20

```
AS   AGE
react-orange-hpa   Deployment/react-orange   cpu: 0%/50%   2         5         2
     16m
react-orange-hpa   Deployment/react-orange   cpu: 0%/50%   2         5         2
     17m
react-orange-hpa   Deployment/react-orange   cpu: 2%/50%   2         5         2
     17m
react-orange-hpa   Deployment/react-orange   cpu: 11%/50%   2         5         2
     17m
react-orange-hpa   Deployment/react-orange   cpu: 30%/50%   2         5         2
     17m
react-orange-hpa   Deployment/react-orange   cpu: 29%/50%   2         5         2
     18m
react-orange-hpa   Deployment/react-orange   cpu: 28%/50%   2         5         2
     18m
react-orange-hpa   Deployment/react-orange   cpu: 34%/50%   2         5         2
     18m
react-orange-hpa   Deployment/react-orange   cpu: 29%/50%   2         5         2
     18m
```

Affinity

- Mecanismo do Kubernetes usado para controlar em quais nodes os pods podem ou devem ser executados
  - separar ambientes (dev, homologação e produção)
  - direcionar aplicações para nodes específicos
  - garantir isolamento
- O Kubernetes verifica os labels dos nodes e compara com as regras definidas no pod

Affinity

- kubectl get nodes
- kubectl label node k3d-meucluster-agent-0 ambiente=dev
- kubectl get nodes --show-labels

```
template:
  metadata:
    labels:
      app: react-blue

  spec:

    affinity:
      nodeAffinity:
        requiredDuringSchedulingIgnoredDuringExecution:
          nodeSelectorTerms:
            - matchExpressions:
                - key: ambiente
                  operator: In
                  values:
                    - dev
    containers:
      - name: react-blue
```

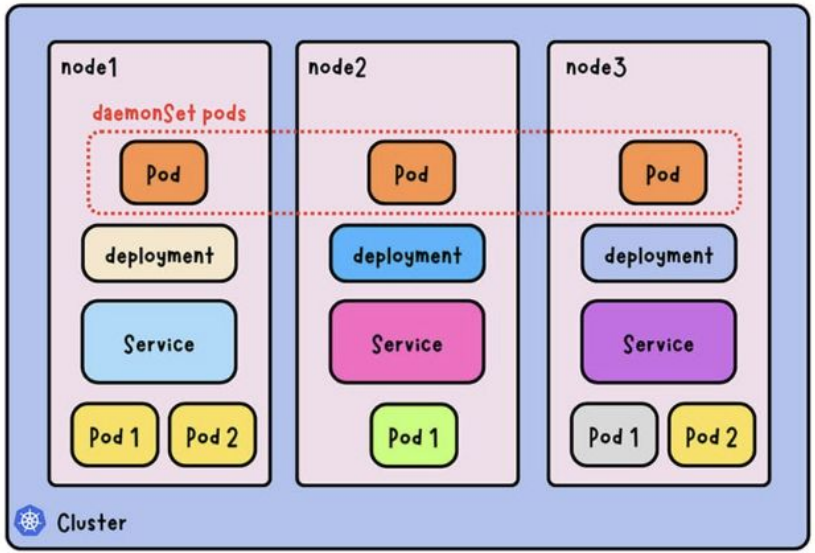
Affinity

```
kubectl get pods -n dev -o wide
NAME                          READY   STATUS    RESTARTS   AGE     IP           NODE                     NOMINATED NODE   READINESS GAT
react-blue-7dbf99cbf8-ww489   1/1     Running   0          2m44s   10.42.2.7    k3d-meucluster-agent-0   <none>           <none>
kubectl get pods -n homolog -o wide
NAME                            READY   STATUS    RESTARTS   AGE     IP           NODE                     NOMINATED NODE   READINESS G
react-orange-86f745d555-n89ht   1/1     Running   0          2m47s   10.42.0.8    k3d-meucluster-agent-1   <none>           <none>
react-orange-86f745d555-x8qll   1/1     Running   0          2m47s   10.42.0.7    k3d-meucluster-agent-1   <none>           <none>
```

- Dessa forma desenvolvimento só vai rodar no node 0 e homologação no node 1

DaemonSets

- Assegura que uma cópia específica de um Pod seja executada em cada nó do cluster
- Diferente de um Deployment, que escala aplicações entre nós, o DaemonSet cria um Pod em cada nó assim que ele entra no cluster



DaemonSets

- Monitoramento
- Gerenciamento de rede
- Segurança
- Métricas

DaemonSet	Deployment
Garante 1 Pod por nó	Distribui réplicas de Pods entre nós
Uso ideal para monitoramento e segurança	Uso ideal para aplicações web ou APIs
Adiciona Pods automaticamente em novos nós	Escalabilidade configurável pelo usuário

StatefulSets

- Deployment
  - stateless
- StatefulSets
  - stateful
    - Banco de dados
  - Garante
    - Um nome fixo e previsível
    - Ordem de inicialização e terminação
    - Volume persistente exclusivo para armazenar dados
    - Não tem IP, resolução de DNS

Característica	StatefulSet	Deployment
Identidade dos Pods	Sim (nomes fixos e persistentes)	Não
Ordem de Inicialização	Garantida e sequencial	Não garantida
Volume Persistente	Volume individual por pod	Compartilhado ou temporário
Uso Comum	Bancos de dados e sistemas distribuídos	Aplicações stateless e APIs
Escalabilidade	Sequencial e controlada	Rápida e flexível
Atualização Rolling Update	Sim (controlada e ordenada)	Sim (rápida)
Persistência de Dados	Alta	Baixa
Recuperação de Pods	Mantém identidade e storage	Cria novo pod genérico
DNS Fixo	Sim	Não
Exemplo de Uso	PostgreSQL, MongoDB, Kafka	Node.js, React, Nginx

StatefulSets

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: "postgres"
  replicas: 2
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
      - name: postgres
        image: postgres:13
        env:
        - name: POSTGRES_USER
          value: "admin"
        - name: POSTGRES_PASSWORD
          value: "secret"
        volumeMounts:
        - name: postgres-storage
          mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
  - metadata:
      name: postgres-storage
    spec:
      accessModes: [ "ReadWriteOnce" ]
      resources:
        requests:
          storage: 2Gi
```

Taints e Tolerations

- Restringir quais pods podem ser executados em determinados nós
  - Taint
    - É uma restrição aplicada a um nó, indicando que certos pods não devem ser agendados nele
      - A menos que tenham uma Toleration correspondente
    - Tipos
      - NoSchedule, PreferNoSchedule e NoExecute

Taints e Tolerations

kubectl taint nodes <nome-do-no> <chave>=<valor>:<efeito>



kubectl taint nodes node1 dedicated=infra:NoSchedule

- Nenhum pod irá executar nesse node sem ter uma tolerância de infra

Taints e Tolerations

- Uma Toleration permite que um pod seja executado em um nó que tenha um Taint correspondente

kubectl taint nodes node1 dedicated=infra:NoSchedule

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: minha-aplicacao
spec:
  replicas: 2
  selector:
    matchLabels:
      app: minha-aplicacao
  template:
    metadata:
      labels:
        app: minha-aplicacao
    spec:
      tolerations:
      - key: "dedicated"
        operator: "Equal"
        value: "infra"
        effect: "NoSchedule"
      containers:
      - name: meu-container
        image: nginx
```

Network Policies

- O tráfego de rede entre os Pods, por padrão, é aberto
- Qualquer Pod pode se comunicar com outro, independentemente do namespace

As Network Policies permitem definir como os Pods podem se comunicar entre si e com outros recursos externos, garantindo segurança e isolamento em ambientes de produção.

Network Policies

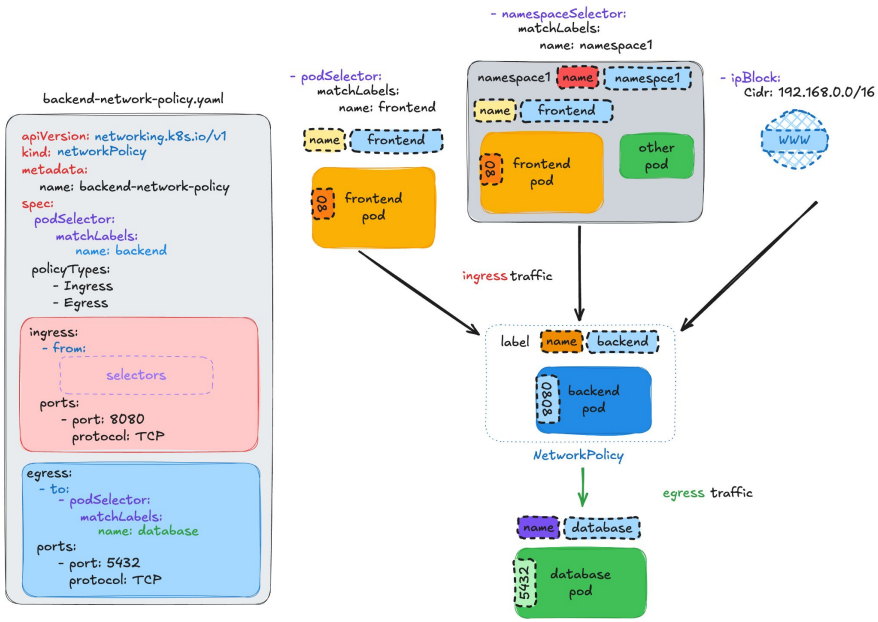
- Tipos
  - Pods de origem: quem está tentando se comunicar
  - Pods de destino: quem vai receber a comunicação
  - Portas e protocolos: por onde e como essa comunicação ocorre
- Elas não têm efeito a menos que o cluster tenha um CNI (Container Network Interface)
  - Calico, Cilium, Weave Net, …

Network Policies

- Estrutura básica
- Pod Selector
  - Define quais Pods a regra se aplica
    - podSelector
    - namespaceSelector,
    - namespaceSelector e podSelector
    - ipBlock
- Ingress
  - Controla o tráfego de entrada

Network Policies

- Ingress
  - Controla o tráfego de entrada
- Egress
  - Controla o tráfego de saída
- Policy Types
  - Define se a política vai aplicar ao tráfego de Ingress ou Egress



Network Policies

```yaml
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: deny-access-all-namespaces
spec:
  podSelector:
    matchLabels:
  ingress:
  - from:
    - podSelector: {}
```

```yaml
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: allow-api-from-frontend
spec:
  podSelector:
    matchLabels:
      app: api
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          role: frontend
```

Segurança

- Uma vez dividindo os namespaces, temos que começar a nos preocupar com os requisitos de segurança do nosso cluster
  - Quem pode acessar o nosso cluster
    - Usuário ou serviços
    - Role-Based Access Control (RBAC)
      - mecanismo do Kubernetes para controlar quem pode acessar o quê dentro de um cluster
  - Contextos de segurança
    - Privilégios

Segurança

- Role
  - Concede permissões específicas dentro de um namespace
- RoleBinding
  - Associa uma Role a um usuário, grupo ou service account em um namespace.
- ClusterRole
  - Define permissões em nível de cluster ou para recursos globais.
- ClusterRoleBinding
  - Associa uma ClusterRole a um usuário, grupo ou service account em todo o cluster

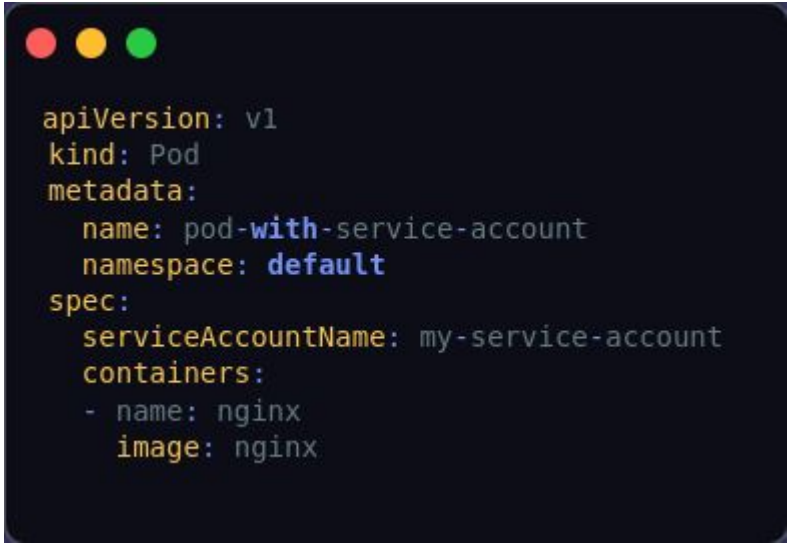
Segurança

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "create"]
```

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods-binding
  namespace: default
subjects:
- kind: User
  name: jane
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Segurança

- Service Accounts
  - São usadas para fornecer uma identidade aos processos que rodam dentro dos Pods
  - Ao contrário das contas de usuário, que são para pessoas, as Service Accounts são projetadas para aplicações e serviços automatizados



```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-service-account
  namespace: default
spec:
  serviceAccountName: my-service-account
  containers:
  - name: nginx
    image: nginx
```

Segurança

- Service Accounts
  - Role
  - Role bind

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
```

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods-binding
  namespace: default
subjects:
- kind: ServiceAccount
  name: my-service-account
  namespace: default
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Segurança

- No Kubernetes, a segurança das aplicações não se limita apenas ao controle de acesso via RBAC
- Mínimo de privilégio possível
  - Security Contexts
    - Níveis
      - Pods
      - Containers



```yaml
apiVersion: v1
kind: Pod
metadata:
  name: secure-pod
spec:
  securityContext:
    runAsUser: 1000
    fsGroup: 2000
  containers:
    - name: secure-container
      image: busybox
      securityContext:
        readOnlyRootFilesystem: true
        allowPrivilegeEscalation: false
      command: ["sleep", "3600"]
```

Segurança

Atributo	Nível	Descrição
`runAsUser`	Pod/Container	Define o UID (User ID) com o qual o processo será executado.
`runAsGroup`	Pod/Container	Define o GID (Group ID) do processo em execução.
`fsGroup`	Pod	Define o GID aplicado aos volumes montados no Pod.
`allowPrivilegeEscalation`	Container	Permite ou impede a elevação de privilégios do processo.
`privileged`	Container	Define se o container será executado em modo privilegiado.
`readOnlyRootFilesystem`	Container	Monta o sistema de arquivos raiz (`root filesystem`) como somente leitura.